

Reference

Acetilsalicylic

April 2, 2026

Contents

1 Template

- 1.1 Acetil's template
- 1.2 Alternative to bits/stdc++.h

2 Language stuff

- 2.1 Primitive types
- 2.2 Useful macros
 - 2.2.1 Limits

3 Algorithms

- 3.1 Binary search
 - 3.1.1 Fast square root
- 3.2 Misc
 - 3.2.1 Binary exponentiation
 - 3.2.2 Mid point calculation
- 3.3 Segment Tree
 - 3.3.1 Recursive
 - 3.3.2 Iterative
 - 3.3.3 Lazy Propagation

4 Data Structures

- 4.1 Ordered Set

5 Maths

- 5.1 Value of pi

1 Template

1.1 Acetil's template

Listing 1: Acetil's template

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 /* num types */
4 typedef long long ll;
5 typedef unsigned long long ull;
6
7 /* data structures */
8 #define mapi map<int, int>
9 #define veci vector<int>
10 #define seti set<int>
11 #define pai pair<int, int>
12 #define F first
13 #define S second
14
15 /* algos */
16 #define fori(n) for (int i=0; i<n; i++)
17 #define fort(t) while(t--)
18 #define fork(k, n) for (int i=k; i<n; i++)
19
20 /* fast IO */
21 #define fastio \
22     ios::sync_with_stdio(0); cin.tie(0)
23
24 /* constants */
25 constexpr int imax = INT_MAX;
26 constexpr ll llmax = LLONG_MAX;
27 constexpr int imin = INT_MIN;
28
29
```

```
30
31 /* solution */
32 void solve() {
33
34 }
35 int main() {
36
37 }
```

1.2 Alternative to bits/stdc++.h

2 Langue stuff

2.1 Primitive types

keyword	Bytes	M_2	m_2	M_{10}	m_{10}
int	4	$2 \cdot 10^9$	$-2 \cdot 10^9$	$2^{31} - 1$	-2^{31}
long long	8	$2^{63} - 1$	-2^{63}	$9 \cdot 10^{18}$	$-9 \cdot 10^{18}$

2.2 Useful macros

2.2.1 Limits

- INT_MAX: Max integer value
- LONG_MAX: Max long int value
- LLONG_MAX: Max long long int value

3 Algorithms

3.1 Binary search

3.1.1 Fast square root

3.2 Misc

3.2.1 Binary exponentiation

Calculate expontents in logarithmic time

3.2.2 Mid point calculation

3.3 Segment Tree

A segment tree is stored as a vector of the datatype it stores. For simple problems, an int type is enough. For more complex or lazy propagation, a Node structure is required.

The segtree is stored in a list, that must have a size of $2N$, where N is the smallest power of two so that $N \geq n$. This is necessary so the binary tree that holds the segtree is complete and can behave correctly.

Elements of a SegTree To build a segment tree, these things are required: an operation, that must be **associative** and preferably, they must have an **identity element**.

Structure Each node of a segtree has a node, and given a node i , it's left child is at index $2i$, and it's right children is at $2i + 1$. The root is at $i = 1$.

Operations The operations on a SegTree usually include the following:

- The index of the node, usually i
- The range over the list, $[l, r]$, where $l \geq 0$; $r < n$.

It's important that the range over the list is used as **zero indexed**, because the original array is zero indexed too.

3.3.1 Recursive

Intuition Each recursive function call represent a current node. This is achieved by passing an i parameter, which is the index of the node inside of the vector representing the tree. Also, two other parameters (tl and tr, standing for temporal low and temporal high respectively) represent the “reach”, or the interval over the original list of elements a that the current node represents (as $[tl, th]$).

Elements The following are the elements required for a recursive segment tree. Each element is defined as following:

- vector<int> tree: The tree itself
- vector<int> a: The original list of elements, read directly from stdin.
- int n: The size of a, the number of elements on the list input.
- int N: The smallest power of two so that $N \geq n$. If $n = 5$, then $N = 8$.
- int len: The actual size of the segtree, $len = 2N$

Listing 2: Recursive segtree declaration stuff - common.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> tree; // size() = len = N * 2
5 vector<int> a; // size() = n
6 int n, N, len;
```

Building is done starting from the root, where $i = 1$. The recursive function descends through every branch, reducing the segment that each node covers (where the interval covered by node i is defined as $[tl, tr]$) in half each time, similar to a binary search.

Listing 3: Recursive build function

```

1 #include "common.cpp"
2
3 /* build the tree */
4 void build(int tl=0, int tr=n-1, int i=1) {
5     // range is [tl, tr]. If tr < tl, it isn't a valid range
6     if (tr < tl) return;
7     // if tl = tr, we've reached a leaf
8     if (tl == tr) {
9         // set the leaves of the tree
10        tree[i] = a[tl];
11        return;
12    }
13
14    int mid = tl + (tr - tl)/2;
15
16    // update the children recursively
17    build(tl, mid, i<<1);
18    build(mid+1, tr, i<<1^1);
19
20    // update parent with new children's values
21    // desired operator instead '+'
22    tree[i] = tree[i<<1] + tree[i<<1^1];
23 }
```

Querying is done combining the top-most nodes that are inside of the range.

Listing 4: Recursive query function

```

1 #include "common.cpp"
2
3 /* query the tree */
4 int query(int l, int r, int tl=0, int tr=n-1, int i=1) {
5     if (tr < l || tl > r)
6         return 0; // Identity element instead of '0'
7     if (tl >= l && tr <= r)
8         // If current node is in range, then return its value
9         return tree[i];
10
11    int mid = tl + (tr-tl)/2;
12
13    // Return the combination of the two halves. This is why
```

```

14    // an identity element is nice: if some half is not valid,
15    // then it won't affect the correct result.
16    //
17    // desired operation instead of '+'
18    return query(l, r, tl, mid, i<<1)
19        + query(l, r, mid+1, tr, i<<1^1);
20 }
```

Updating a position implies doing some binary-search-like traversal and updating a leaf and its parents.

Listing 5: Recursive update function

```

1 #include "common.cpp"
2
3 /* update a position */
4 void update(int pos, int val, int tl=0, int tr=n-1, int i=1) {
5     if (tr < tl) return;
6     if (tr < pos || tl > pos) return;
7     if (tl == tr) {
8         if (tl != pos) return;
9         tree[i] = val;
10        return;
11    }
12
13    int mid = tl + (tr - tl)/2;
14
15    update(pos, val, tl, mid, i<<1);
16    update(pos, val, mid+1, tr, i<<1^1);
17
18    // Desired operation instead of '+'
19    tree[i] = tree[i<<1] + tree[i<<1^1];
20 }
```

3.3.2 Iterative

3.3.3 Lazy Propagation

4 Data Structures

4.1 Ordered Set

Exclusive of the GCC compiler collection.

Listing 6: PBDS ordered set declaration

```

1 // Common file
```

```

2 #include <ext/pb_ds/assoc_container.hpp>
3 // Including tree_order_statistics_node_update
4 #include <ext/pb_ds/tree_policy.hpp>
5
6 using namespace std;
7 using namespace __gnu_pbds;
8
9 typedef tree<
10     int ,
11     null_type ,
12     less<int> ,
13     rb_tree_tag ,
14     tree_order_statistics_node_update
15 > ordered_set;

```

5 Maths

5.1 Value of pi

The value of π is not always available. It can be obtained as $\cos^{-1}(-1)$.